

SOFTWARE PRODUCT STUDIO • ADVANCED CAPSTONE

Ship a product that **proves** your skills.

Turn a real problem into a focused, usable, tested release — then tell the evidence-led story behind it.



A capstone is **evidence**, not a feature pile.

THE WEAK PATTERN

“I built many screens.”

A long list of disconnected functions, unclear users, unfinished states, and no proof that the product helps anyone.

THE CREDIBLE PATTERN

“A user can complete one meaningful outcome.”

A focused problem, working core journey, observed feedback, and a clear explanation of the decisions behind the release.

01

Problem

Who feels it and why it matters.

02

Scope

What the first release will not do.

03

Use

What a real person can complete.

04

Learning

What testing changed in the build.

Start with a problem someone actually feels.

Great products don't start with a technology stack. They start with an observed friction in a specific person's day. If you cannot name the user and their pain point, you are not ready to write code.

THE PROBLEM SIGNAL CHECKLIST

01 Observe the friction

Watch a user struggle with their current tool or process. Don't just ask what they want; see what they actually do.

02 Narrow the audience

Define the specific person experiencing this. "Everyone" is not a target audience for a first release.

03 Describe the gap

Write down the exact difference between what the user is trying to achieve and what their current tools allow.

04 Choose a measurable outcome

Decide how you will know if the friction is reduced (e.g., "Time to complete task drops from 5 minutes to 1 minute").

Turn observations into a **testable brief**.

THE CORE HYPOTHESIS

User

Who exactly is experiencing this friction?

Context

When and where does this problem occur?

Problem

What is the specific pain point or blocker?

Outcome

What does success look like for the user?

THE BOUNDARIES

Non-goals

What are we explicitly NOT building in this release?

Success Signal

How will we measure if the solution actually worked?

Constraints

What technical, time, or resource limits exist?

Scope the **smallest product** that proves the promise.

THE VERTICAL SLICE

Do not build the entire database, then the entire API, then the entire UI. **Build the UI, logic, and data for one single user action** from start to finish before moving to the next.

Prioritise the Core Journey

Identify the one path the user absolutely must complete for the product to have value. Build this primary flow first, ignoring edge cases.

Define Strict Non-Goals

Explicitly list what you will not build. "We will not build user profiles. We will not build password reset." Protect your scope by saying no.

Protect the Build Plan

A capstone is strictly timeboxed. Scope creep is the enemy of a finished product. If a feature takes longer than a week, cut it immediately.

Map the journey before writing the **components**.

Code is expensive to rewrite. A structural map ensures you only build what the user actually needs to complete the core journey.

01 / ACTION

User Flow

Define the exact steps the user takes to reach the outcome.

Click 'Book'



Select Date

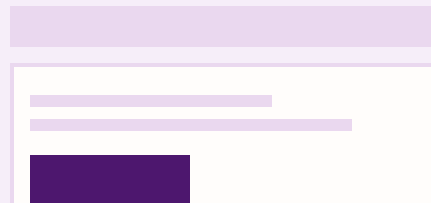


Confirm

02 / LAYOUT

Screen Sketch

Block out the information hierarchy and interactive zones.



03 / PAYLOAD

Data Map

Identify the exact JSON shape required by the interface.

```
{
  "id": "req_92",
  "date": "2026-10",
  "status": "active"
}
```

04 / EDGE CASES

System States

Plan for the moments when the happy path breaks.

Loading...

Error: Network

Success

Build from a **delivery board**, not a wish list.

A capstone fails when you try to build everything at once. Organize work into strict timeboxes and define exactly what "done" means before writing code.

NOW (ACTIVE)

User can submit booking form

ACCEPTANCE CRITERIA

- Validates email format
- Saves payload to database
- Shows success confirmation

Risk: DB Connection

NEXT (UP DECK)

User can view past bookings

ACCEPTANCE CRITERIA

- Fetches user ID from session
- Renders list in chronological order

Blocked by: Booking Form

LATER (WISH LIST)

Email notifications

STATUS

- Out of scope for MVP
- Requires 3rd party API setup

Ship a **usable increment** before polishing.

Do not spend three days perfecting CSS on a form that cannot submit data. Build the ugly, working version first. Prove the data flows, handle the errors, and only then make it beautiful.

PHASE 01

Entry Point

Build the raw HTML structure. A basic button or unstyled form that captures the necessary user input.

PHASE 02

Data Path

Connect the frontend to the backend. Ensure the payload reaches the database and returns a valid response.

PHASE 03

Feedback States

Implement loading indicators and error messages so the user is never left wondering what the system is doing.

PHASE 04

Completed Action

Handle the success state. Clear the form, update the UI with the new data, and apply final visual polish.

Test the product and record what changed.

Testing isn't just finding bugs; it's proving your product works for real people. A strong portfolio shows exactly how user feedback directly improved your code.

STEP 01

Prepare Tasks

Give the user a goal (e.g., "Book a session for tomorrow"), not step-by-step instructions on where to click.

STEP 02

Observe Behaviour

Watch silently. Note where they hesitate, scroll aimlessly, or click the wrong elements. Do not help them.

STEP 03

Capture Evidence

Write down the exact friction point. "User didn't see the submit button because it was below the fold."

STEP 04

Prioritise Fixes

Fix critical blockers immediately. Ignore minor visual polish if the core journey is still broken.

STEP 05

Rerun Checks

Deploy the fix and test the exact same task with a new user to verify the friction is gone.

THE OUTCOME

Portfolio Value

Documenting this loop proves to employers that you build software for users, not just for yourself.

Deployment means **another person** can use it.

"It works on my machine" is not a release. A product is only finished when a stranger can open it, understand what it does, and use it without your help.

ENVIRONMENT

- Live URL is publicly accessible
- Environment variables are secured
- Production database is connected and seeded

RESILIENCE

- 404 and error states are handled gracefully
- Basic keyboard navigation works
- Core journey is usable on mobile screens

HANDOVER

- README explains the project purpose
- Local setup steps are clearly documented
- Test credentials are provided for reviewers

Your portfolio explains **decisions and evidence.**

A link to a GitHub repository is not a portfolio. A strong case study tells the story of why you built the product, what you chose to cut, and how you proved it worked.

PART 01

The Problem Context

Start with the user. Clearly define the specific friction or pain point you observed, and state your core hypothesis for how software could solve it.

PART 02

Scope & Trade-offs

Explain your vertical slice. Document what you explicitly chose **not** to build, demonstrating your ability to prioritize the core journey over feature bloat.

PART 03

Execution & Evidence

Show the architecture, but more importantly, show the test results. Provide evidence of user testing, the friction you uncovered, and how you fixed it.

PART 04

The Next Iteration

No product is ever truly finished. Conclude by reflecting on what you learned technically, and what the next logical feature would be based on user data.

Deliver the first **credible release**.

Execute a complete product cycle from problem definition to a live, usable MVP. This is your opportunity to prove you can ship software that solves real problems.

REQUIRED ARTEFACTS

- Product Brief (Problem, User, Scope boundaries)
- Delivery Board (Now, Next, Later prioritization)
- Live URL of the deployed MVP
- Documented User Test (1 friction point observed & fixed)
- GitHub Repository with a professional README

QUALITY BAR

- The core journey works end-to-end without crashing.
- Scope is strictly limited to the primary value proposition.
- A stranger can set up the project using only the README.

EXTENSION CHALLENGE

The Portfolio Case Study

Code alone doesn't get you hired; explaining your decisions does. Write a 500-word case study to accompany your live link.

Explain **why** you built it, **what** you explicitly cut from the scope to meet the deadline, and **how** user testing changed your initial design.

Publish this case study on a blog, Notion, or personal site, and link it directly from your GitHub README.